CAPACITACIÓN QA AUTOMATION · PARTE 2 · SESIÓN 3 DE 4

Screenplay + Cucumber a escala

El actor vive en el World y los steps solo delegan. Tareas compuestas, tablas de datos y evidencia útil cuando algo falla: captura, traza y reportes.

Instructor **Giancarlo Palomino**

Jueves 29 de octubre · 17:30 a 18:45

Playwright 1.64 · Cucumber 13 · TypeScript

75 MINUTOS

De un login a una tienda completa

20 min — **El actor vive en el World**
Los steps solo delegan; tareas compuestas y tablas de datos.

15 min — **Evidencia**
Captura y traza de Playwright solo cuando falla; reportes HTML, JSON y JUnit de Cucumber.

40 min — **Ejercicios 4 y 5**
Una compra completa que cuadra · Caza de bugs con `problem_user`. Entrega: PR `feature/s3-<tu-nombre>` con `BUGS.md`.

REQUISITO

El ejercicio 3 en verde: el núcleo Screenplay (`Actor` , `NavegarLaWeb` , `Clic` , `Escribir` …) ya existe en tu rama.

AL TERMINAR SABRÁS

Escalar Screenplay a flujos de varias pantallas sin que los steps crezcan, y entregar evidencia que cualquiera puede abrir.

UN ACTOR NUEVO POR ESCENARIO

El actor vive en el World

src/support/world.ts — WORLD

```ts
// El World es el contexto que comparten los steps de un mismo escenario.
// Ahora también guarda al actor: los steps solo hablan con él.
export class CustomWorld extends World {
  context!: BrowserContext;
  page!: Page;
  actor!: Actor;
}
```

src/support/hooks.ts · Before — HOOK

```ts
Before(async function (this: CustomWorld) {
  this.context = await browser.newContext({ baseURL: BASE_URL });
  await this.context.tracing.start({ screenshots: true, snapshots: true });
  this.page = await this.context.newPage();
  this.actor = Actor.llamado('Comprador')
    .puede(NavegarLaWeb.con(this.page));
});
```

BEFOREALL · UNA VEZ

chromium.launch()

BEFORE · CADA ESCENARIO

contexto + traza + página + actor

nadie hereda la sesión ni las notas de otro

STEPS

this.actor.intenta / pregunta

nunca `this.page`

AFTER · CADA ESCENARIO

evidencia si falló + cerrar contexto

POR QUÉ EN EL WORLD

Cucumber crea un World nuevo por escenario: el actor, sus habilidades y lo que anota mueren con el escenario. Independencia gratis.

TRADUCIR, NO PROGRAMAR

Un step es una línea: delegar en el actor

STEP CON LÓGICA · EVITAR

carrito.steps.ts ✗

```
When('agrego al carrito los productos:',
  async function (this: CustomWorld, tabla: DataTable) {
    for (const [nombre] of tabla.raw()) {
      await this.page
        .locator('[data-test="inventory-item"]', { hasText: nombre })
        .getByRole('button', { name: 'Add to cart' })
        .click();
    }
  });
```

Selectores y bucles en el step. El próximo escenario que agregue productos lo copiará.

STEP QUE DELEGA · ASÍ

src/steps/carrito.steps.ts ✓

```
// Tabla de una columna: raw() devuelve
// [['Sauce Labs Backpack'], ['Sauce Labs Bike Light'], ...]
When('agrego al carrito los productos:',
  async function (this: CustomWorld, tabla: DataTable) {
    await this.actor.intenta(
      AgregarAlCarrito.losProductos(...tabla.raw().flat()));
  });

When('quito {string} del carrito',
  async function (this: CustomWorld, producto: string) {
    await this.actor.intenta(QuitarDelCarrito.elProducto(producto));
  });
```

El step convierte el texto en datos y llama a una tarea. Nada más.

PRUEBA RÁPIDA

Busca `page`, `locator` o `data-test` en `src/steps/`. Si aparece, algo está en la capa equivocada.

EJERCICIO 4 POR DENTRO

CompletarCompra: el flujo de negocio, leído en voz alta

src/screenplay/tareas/CompletarCompra.ts · extracto — TASK

```ts
export const RESUMEN = 'resumen de compra';

export class CompletarCompra implements Performable {
  private constructor(private readonly datos: Datos) {}

  static con(datos: Datos): CompletarCompra {
    return new CompletarCompra(datos);
  }

  async realizadaPor(actor: Actor): Promise<void> {
    await actor.intenta(
      IrAlCarrito.yPasarAlCheckout(),
      IngresarDatosDeEnvio.con(this.datos),
      Clic.en(Checkout.CONTINUAR),
      Anotar.laRespuestaA(ResumenDeCompra.enPantalla()).como(RESUMEN),
      Clic.en(Checkout.FINALIZAR),
    );
  }
}
```

```
CompletarCompra            Task
├─ IrAlCarrito             Task
│  ├─ Clic ícono carrito   Interaction
│  └─ Clic checkout        Interaction
├─ IngresarDatosDeEnvio    Task
│  └─ Escribir × 3         Interaction
├─ Clic continuar          Interaction
├─ Anotar el resumen       Interaction
└─ Clic finalizar          Interaction
```

REUTILIZABLE

`IrAlCarrito` e `IngresarDatosDeEnvio` también los usan el caso negativo y la caza de bugs.

CUANDO LA PANTALLA DESAPARECE ANTES DEL ENTONCES

El actor anota lo que vio y lo recuerda después

src/screenplay/interacciones/Anotar.ts · extracto — INTERACTION

```ts
static laRespuestaA<T>(pregunta: Question<T>) {
  return { como: (clave: string) => new Anotar(pregunta, clave) };
}

async realizadaPor(actor: Actor): Promise<void> {
  actor.anota(this.clave, await actor.pregunta(this.pregunta));
}
```

src/screenplay/preguntas/ResumenDeCompra.ts · extracto — QUESTION

```ts
async respondidaPor(actor: Actor): Promise<Resumen> {
  return {
    subtotal: aNumero(await actor.pregunta(
      TextoDe.el(Checkout.SUBTOTAL))),
    impuesto: aNumero(await actor.pregunta(
      TextoDe.el(Checkout.IMPUESTO))),
    total: aNumero(await actor.pregunta(TextoDe.el(Checkout.TOTAL))),
    precios: await actor.pregunta(Precios.de(Checkout.PRECIOS)),
  };
}
```

src/steps/compra.steps.ts · extracto — THEN

```ts
Then('el resumen muestra subtotal, impuesto y total correctos',
  async function (this: CustomWorld) {
  const { precios, subtotal, impuesto, total } =
    this.actor.recuerda<Resumen>(RESUMEN);
  const sumaDePrecios = precios.reduce((suma, precio) => suma + precio, 0);
  expect(subtotal, 'subtotal = suma de los precios')
    .toBeCloseTo(sumaDePrecios, 2);
  expect(impuesto, 'impuesto = 8 % del subtotal')
    .toBeCloseTo(Math.round(subtotal * 8) / 100, 2);
  expect(total, 'total = subtotal + impuesto')
    .toBeCloseTo(subtotal + impuesto, 2);
});
```

DATO VERIFICADO

Mochila $29.99 + luz $9.99 → subtotal **$39.98**, impuesto **$3.20**, total **$43.18**.

DATATABLE DE CUCUMBER

Dos formas de tabla, dos métodos

Una columna · lista — RAW()

```
Cuando agrego al carrito los productos:
  | Sauce Labs Backpack     |
  | Sauce Labs Bike Light   |
  | Sauce Labs Bolt T-Shirt |
```

Dos columnas · clave y valor — ROWSHASH()

```
Cuando completo la compra con los datos:
  | nombre       | Ana    |
  | apellido     | Torres |
  | codigoPostal | 15001  |
```

en el step — TYPESCRIPT

```typescript
tabla.raw()        // [['Sauce Labs Backpack'], ...]
tabla.raw().flat() // ['Sauce Labs Backpack', ...]
```

src/steps/compra.steps.ts — TYPESCRIPT

```typescript
// Tabla de dos columnas (clave | valor): rowsHash() la convierte en un objeto.
const datosDe = (tabla: DataTable) => tabla.rowsHash() as unknown as Datos;
```

Y SI LA TABLA TIENE ENCABEZADO

`tabla.hashes()` devuelve un objeto por fila: `[{ producto: 'Backpack', cantidad: '1' }, ...]`. Todo llega como texto: convierte números tú.

WEB-FIRST CON SCREENPLAY

Una pregunta devuelve una foto; `expect.poll` la repite

src/screenplay/preguntas/ContadorDelCarrito.ts · extracto — QUESTION

```ts
async respondidaPor(actor: Actor): Promise<number> {
  const contador = NavegarLaWeb.de(actor)
    .localizar(Catalogo.CONTADOR_CARRITO);
  if (!(await contador.isVisible())) return 0;
  return Number(await contador.innerText());
}
```

src/steps/carrito.steps.ts · extracto — THEN

```ts
// expect.poll repite la pregunta hasta que la respuesta cumple
// (o vencen 5 s): aserción web-first.
Then('el carrito indica {int} producto(s)',
  async function (this: CustomWorld, cantidad: number) {
    await expect.poll(() =>
      this.actor.pregunta(ContadorDelCarrito.enLaCabecera()))
      .toBe(cantidad);
  });
```

PREGUNTA	¿ESPERA SOLA?	ÚSALA CON
`TextoDe`	Sí: `innerText()` espera al elemento	`expect(...)`
`TextosDe`, `CantidadDe`	No: devuelven lo que hay ahora	`expect.poll`
`EsVisible`, `ContadorDelCarrito`	No	`expect.poll`

LA REGLA DE SIEMPRE

Nada de `waitForTimeout`. Si el valor puede tardar en cambiar, la aserción reintenta.

02

Evidencia

Cuando un escenario falla, el reporte tiene que explicar qué pasó sin que nadie lo vuelva a ejecutar.

SOLO CUANDO FALLA

La traza se graba siempre y se guarda solo si hace falta

src/support/hooks.ts · After — HOOK

```ts
// Evidencia solo cuando falla: captura + traza en reports/ y adjuntas al reporte.
After(async function (this: CustomWorld, { pickle, result }) {
  if (!this.context) return;
  if (result?.status === Status.FAILED) {
    const nombre = `${aArchivo(pickle.name)}-${pickle.id.slice(0, 8)}`;
    const captura = await this.page.screenshot({
      path: `reports/capturas/${nombre}.png`, fullPage: true });
    this.attach(captura, { mediaType: 'image/png', fileName: `${nombre}.png` });

    const traza = `reports/trazas/${nombre}.zip`;
    await this.context.tracing.stop({ path: traza });
    this.attach(await readFile(traza),
      { mediaType: 'application/zip', fileName: `${nombre}.zip` });
    this.attach(`Para ver la traza: npx playwright show-trace ${traza}`, 'text/plain');
  } else {
    await this.context.tracing.stop();
  }
  await this.context.close();
});
```

BEFORE

`context.tracing.start({ screenshots: true, snapshots: true })` : graba cada acción con el DOM de antes y de después.

PASÓ

`tracing.stop()` sin ruta: no se escribe nada. El verde no ocupa disco.

FALLÓ

Captura de página completa y traza `.zip` en `reports/`, ambas adjuntas al reporte HTML.

```
$ npx playwright show-trace
reports/trazas/<archivo>.zip
```

LA TRAZA

Línea de tiempo, cada clic con su captura, la red y la consola. Es la mejor herramienta para depurar un rojo de la CI.

UN REPORTE PARA CADA LECTOR

HTML para personas, JSON y JUnit para herramientas

cucumber.js · extracto — CONFIGURACIÓN

```
const comun = {
  paths: ['features/**/*.feature'],
  import: ['src/**/*.ts'],
  format: [
    'progress-bar',
    'html:reports/cucumber-report.html',
    'json:reports/cucumber-report.json',
    'junit:reports/cucumber-report.xml',
  ],
  formatOptions: { snippetInterface: 'async-await',
    junit: { suiteName: 'SauceDemo' } },
  language: 'es',
};
```

ARCHIVO	PARA QUIÉN
`cucumber-report.html`	Personas: escenarios, pasos, duración, captura y traza adjuntas. Doble clic y listo.
`cucumber-report.json`	Herramientas de reportes y tableros (por ejemplo, para métricas históricas).
`cucumber-report.xml`	JUnit: lo entienden GitHub Actions, Jenkins y Azure DevOps. Mañana lo usamos en el PR.
`capturas/`, `trazas/`	Solo existen si algo falló.

REPORTS/ NO SE SUBE A GIT

Está en `.gitignore`. En la CI se publica como artefacto.

35 MINUTOS DE BLOQUE 1 Y 2

Guion de la demo

1	0–4	Mostrar `world.ts` y el `Before`: el actor nace con cada escenario.
2	4–10	Migrar el carrito de ayer: `AgregarAlCarrito`, `QuitarDelCarrito`, `ContadorDelCarrito` con `expect.poll`.
3	10–16	Escribir `IrAlCarrito` e `IngresarDatosDeEnvio`; armar `CompletarCompra` en vivo. `rowsHash()`.
4	16–20	El problema del resumen que desaparece: `Anotar` y `recuerda`.
5	20–26	Romper a propósito: cambiar `'Products'` por `'Productos'` y correr. Abrir el HTML: captura y traza adjuntas.
6	26–32	`npx playwright show-trace`: recorrer la línea de tiempo. Agregar JSON y JUnit a `cucumber.js`.
7	32–35	Deshacer el error, `npm test` en verde: `reports/` sin capturas ni trazas.

```
$ npm test -- --tags @ejercicio4
2 scenarios (2 passed)
$ npm test
20 scenarios (20 passed)
115 steps (115 passed)
```

ANTES DE CLASE

Tener abierto el reporte con una falla ya generada por si la red de la sala está lenta.

4
NIVEL

DE PUNTA A PUNTA · CON SCREENPLAY

Una compra completa que cuadra

20 min
Dificultad ●●●●○

OBJETIVO

Catálogo → carrito → datos de envío → resumen → confirmación con una tarea compuesta `CompletarCompra`, y preguntas que comprueban que **los montos cuadran**.

CRITERIOS DE ACEPTACIÓN

1. `CompletarCompra` compone otras tareas e interacciones.
2. Datos del comprador por tabla de dos columnas (`rowsHash()`).
3. Subtotal = suma de precios · impuesto = 8 % · total = subtotal + impuesto.
4. Confirmación: *Thank you for your order!*
5. Negativo: sin datos, error *First Name is required* y no avanza.

Aprendes: tareas que componen tareas, la memoria del actor y cálculos en las aserciones.

features/compra.feature — GHERKIN

```gherkin
@smoke
Escenario: Comprar dos productos
  Dado que agregué al carrito "Sauce Labs Backpack" y "Sauce Labs Bike Light"
  Cuando completo la compra con los datos:
    | nombre       | Ana    |
    | apellido     | Torres |
    | codigoPostal | 15001  |
  Entonces el resumen muestra subtotal, impuesto y total correctos
  Y veo la confirmación "Thank you for your order!"

Escenario: No se puede pagar sin datos de envío
  Dado que agregué al carrito "Sauce Labs Backpack"
  Cuando intento pagar sin datos de envío
  Entonces veo el mensaje de error "First Name is required"
  Y sigo en el paso de datos de envío
```

PISTAS

`data-test` : `checkout` , `firstName` , `lastName` , `postalCode` , `continue` , `finish` , `subtotal-label` , `tax-label` , `total-label` , `complete-header` .

5
RETO

PENSAR COMO QA

Caza de bugs con `problem_user`

20 min
Dificultad ●●●●●

OBJETIVO

Correr las verificaciones de los ejercicios 2 a 4 con `problem_user` y documentar lo que se rompe: un escenario que lo demuestra y un reporte que cualquiera puede reproducir.

CRITERIOS DE ACEPTACIÓN

1. Al menos tres bugs, cada uno con un escenario que falla por la causa correcta.
2. El usuario se parametriza: la misma verificación pasa con `standard_user`.
3. Lo que falla por un bug lleva `@bug`, y `npm test` **sigue en verde**.
4. `BUGS.md`: título, usuario, pasos, esperado, obtenido, captura y severidad.

PISTAS, SIN SPOILERS

1. Mira con atención las imágenes del catálogo.
2. Tu ejercicio 3 ya sabe si el ordenamiento de verdad ordena.
3. ¿Todos los botones *Add to cart* hacen algo?
4. En el checkout, ¿lo que escribes queda en su campo?
5. ¿Cuánto tarda en entrar `performance_glitch_user`? ¿Tus pruebas lo toleran?

ENTREGABLE

PR `feature/s3-<tu-nombre>` con los ejercicios 4 y 5, `BUGS.md` y el comando que corre los `@bug` en la descripción.

Aprendes: que una buena aserción encuentra bugs sin buscarlos, y a reportarlos.

LEER EL ROJO

Seis tropiezos de hoy

```
Comprador no anotó "resumen de compra"
```

Recordar sin anotar

El *Entonces* pide algo que ninguna tarea anotó, o con otra clave. Usa constantes (`RESUMEN`).

```
Expected: 3 · Received: 2
```

Aserción sin reintento

El contador todavía no se actualizó. Una pregunta que no espera va con `expect.poll` .

```
Expected: 3.2 · Received: 3.1984
```

Decimales

Redondea como la app (2 decimales) y compara con `toBeCloseTo(x, 2)` .

```
Ambiguous step definitions
```

Dos steps para la misma frase

El de ayer (Page Object) y el de hoy (Screenplay) a la vez. Al migrar, borra el viejo.

```
0 scenarios
```

Filtro que no encuentra nada

`npm test -- --tags @bug` se combina con `not @bug` . Para los bugs: `npm run test:bugs` .

```
reports/ de 50 MB
```

Traza siempre guardada

Guardar la traza también en verde llena el disco y la CI. Solo cuando falla.

CANVIA

PARA LLEVARTE · ENTREGA: PR FEATURE/S3-<TU-NOMBRE> CON BUGS.MD

Escenarios cortos, tareas que escalan, evidencia que explica

1 · Steps de una línea
El actor en el World; el step solo traduce y delega.

2 · Rojo que se explica
Captura y traza solo cuando falla, adjuntas al reporte.

3 · Mañana
Todo esto en GitHub Actions, el reporte publicado en Pages y el examen final.

Tarea: ejercicios **4 y 5** con **npm test** en verde

Mañana: tu PR con la CI en verde